

Checkpoint 3

Due: March 16th, by 11:59pm

Overview

Before starting on this checkpoint, it is assumed that you have a working renderer module which can render an *entire*, single frame of animation based on the state of the model's data structures. This checkpoint will have you implementing, the input module, the main game loop and all of the game logic for the 1-player game, and then once that is completed you will implement double-buffering

This Checkpoint Takes Time

This checkpoint takes a significant amount of time as **ALL** game logic will be implemented as part of this checkpoint. Completing this checkpoint well is vital as the rest of the checkpoints require that the game logic is completed to be able to do subsequent checkpoints.

Ideally, if you have completed *Minimizing Screen Plotting* portion of Checkpoint 2, the renderer can also render an *updated* frame of animation (i.e. without redrawing the entire frame), based on information about which elements of the game model have changed since a previous invocation of the renderer.

In this checkpoint you will create a main game module. The module will contain a main game loop which polls for and detects clock ticks and keyboard input. These will allow the model to evolve in response to synchronous and asynchronous events, respectively. The main game loop will also periodically invoke the renderer, so that a sequence of frames of animation are produced over time. In this stage your game world ceases to be static, and becomes dynamic. In other words, your game will start to become playable!

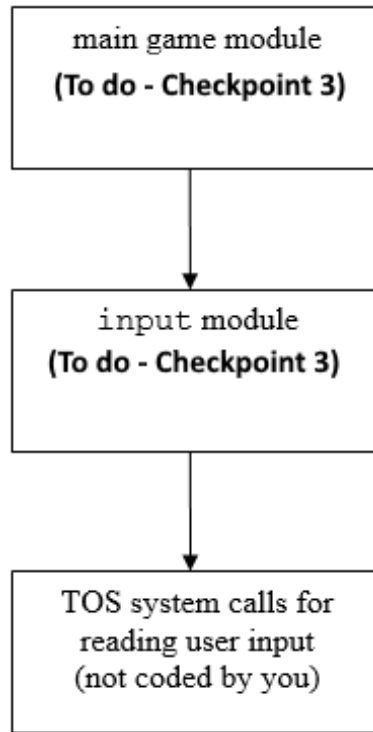
If you haven't yet completed *Minimizing Screen Plotting*, it is still possible to make progress on the main game module; however, the animation will be unnecessarily slow and choppy. Therefore, you will be forced to refine your renderer so that plotting is minimized. At the very latest, it will be necessary to address this issue prior to implementing double buffering.

Regardless of how well you complete the renderer, minimized screen plotting, and the main game loop, the animation will remain imperfect. You will probably notice graphical *flickering* and *tearing* effects. In short, this will be because plotting will not be synchronized with the screen refresh rate of the video hardware. You will fix this problem using a technique called *double buffering*.

INPUT MODULE

Background: Three-Level Input Design

When you code the user input portion of your game, you must follow a three-level design:



The middle layer contains the *high-level* input routines needed by the game. These in turn call the *low-level* input routines provided by the operating system. The former are hardware-independent, whereas the latter deal directly with the input hardware.

In summary, identify the input routines needed by your game and then implement a corresponding high-level input library named *input* with corresponding C files *input.h* and *input.c*. Note that these library routines must actually be game independent (i.e. it should be possible to reuse the library in other applications). These routines will wrap underlying, O/S-specific code for performing keyboard and mouse input. This will ensure that game code is as O/S-independent as possible.

The advantage of this approach will become apparent in later stages, when the O/S support will be replaced by custom low-level code. If the main game module code has not been decoupled from the O/S, it will make removal of the O/S support difficult. Instead, it should be possible to leave the game code alone and to only modify the middle layer wrappers.

Like for the previous stages, you must provide one or more test driver programs which invoke the middle layer routines.

Requirement: Input Module

You must create an input module. The input module will consist of two files `input.h` and `input.c`. The minimum functions required are a *has_input* function which returns true if there is pending input to be processed and false otherwise, and a *get_input* function which returns the next character to be processed.

Main Module & System Calls

Since the input module is the layer that communicates with the operating system for user input there must be **NO** system calls or functions wrapping system calls, related to user input, in the main game module!

ANIMATION AND GAME LOGIC

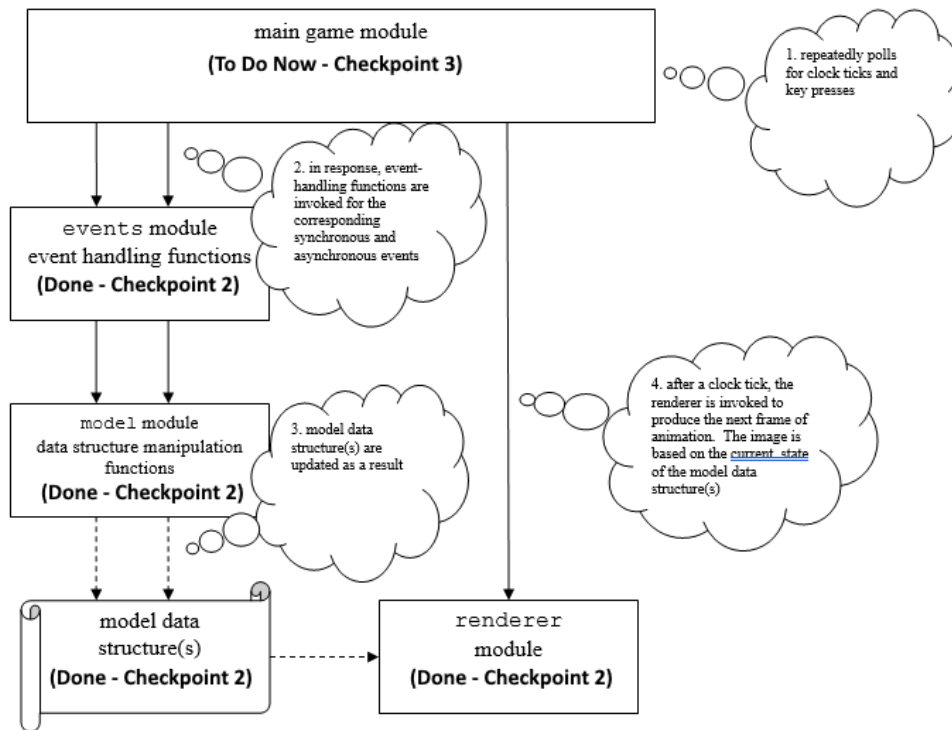
Background: The Design of Video Games, Part 2

In an animated video game, individual frames of animation are rendered in real time based on the current state of the model. The “frame rate” is the rate at which successive frames are displayed. The faster the frame rate, the smoother the animation will appear to be. For this game, frame rates up to 70 fps (frames per second) are achievable¹. This should appear smooth to most human observers. By contrast, frame rates down to about 12 fps may be perceived by most humans as animated, but jumpy².

A clock is used to drive the frame rate. It can also be used to produce synchronous (internally generated, clock synced) events within the game world. Of course, asynchronous (externally generated, non-clock synced) events such as user input must also be dealt with. Each event will typically entail updating some aspect of the model. Consequently, the next time a frame is rendered, the view of the world may have changed.

¹The Atari ST’s monochrome monitor has a 70 Hz refresh rate.

²Some commercial video games have frame rates in the 25-35 fps range. This is considered only minimally acceptable by some players. Other games attempt to match the refresh rate of the monitor, which is around 85 Hz on an average modern monitor. Still other games have an even higher frame rate, even though the monitor can’t match it. In this case, the extra frames are discarded (or the frames aren’t synchronized with the refresh rate, which leads to visual disturbances such as *tearing*).



An essential part of a game's code is its main game loop, found within the main game module. The algorithmic structure of this module is illustrated below:

```

initialize model
render model (first frame)
set quit = false

repeat until quit
  if input is pending
    process async event      <-- update model change requests
  if clock has ticked
    process sync events      <-- update model data
    process cond events      <-- update model data
  render model (next frame)
  
```

Note: Not every iteration of the loop will have an action being performed, as there may not be any unprocessed input nor may a clock tick have occurred.

Checkpoint Requirement

You are **REQUIRED** to use this main loop algorithm, without structural changes! It is critical to understand that asynchronous events update the model's request fields, i.e. pressing the up arrow key will set the direction to up. Pressing the up arrow **DOES NOT** move the item up. It is only on a clock tick that **ALL** game items are moved, i.e. data or position fields. Failure to follow this design will result in you needing to come back to this stage later in the semester and completely re-code from this stage onwards.

Requirement: Main Game Module

You must create a main game module. This must be coded as a source code file with an appropriate game-specific name (e.g. *pong.c*). This module must contain your game's main function. It may also contain various helper functions. Your makefile must generate a game-specific executable (e.g. *pong* or *pong.tos*).

The main game module must contain game initialization code, as well as the main game loop. The game is not to display welcome menus or splash screens at this stage.

TOS maintains a 70 Hz clock at memory location 0x462. Every 1/70th of a second, the longword value at this address is auto-incremented. The main game loop must repeatedly poll this value. Each time it changes synchronous events can be triggered. The rendering of the next frame of animation must also be triggered by clock ticks. You must not use user input events as rendering triggers!

To the main game loop, you must also add checks for user input so that asynchronous events can be triggered.

Background: How to Detect Clock Ticks

Important: The TOS maintains a 70 Hz timer: a longword at address 0x462 which is auto-incremented 70 times per second. Your game can monitor the passage of time by periodically reading the contents of this memory location.

The timer memory location is protected: the ST hardware monitors memory accesses at this address, and prevents them unless the CPU is in a "privileged" mode. Normally, only TOS code runs in this privileged mode, while ordinary user programs that attempt to access the timer crash with a bus error:

```
long *timer = (long *)0x462; /* address of longword auto-inc'd 70 x per s */
long timeNow;

...

timeNow = *timer;           /* crashes with bus error (2 bombs) here! */

...
```

However, TOS allows³ ordinary user programs to run in privileged mode! All that is required is a call to the `Super` system call:

```
#include <osbind.h>

...

long old_ssp;

old_ssp = Super(0);  /* enter privileged mode */
timeNow = *timer;
Super(old_ssp);      /* exit privileged mode as soon as possible */

...
```

The above code should be incorporated into a helper function called `get_time` which takes no parameters and returns the current timer value as a `UINT32`.

Aside: While it may seem like a good idea to run your entire program in privileged mode it is actually a terrible idea. When in privileged mode you have unfettered access to everything. Thus, if you inadvertently alter a system variable or peripheral the operation of the system can change, possibly causing the system to hang or crash. For example disabling the keyboard will result in the system appearing to hang. In non-privileged mode this action would have resulted in 8 bombs indicating a Privileged Violation. In privileged mode there will not be any bombs and you will have no idea what is the problem and where it occurred or why. Thus, it is common to change into privileged mode, perform only those actions that require privileged access and then leave privileged mode.

Once the above is working, it should be possible to incorporate code like the following into the main game loop:

```
UINT32 timeThen, timeNow, timeElapsed;

...

timeNow = get_time();
timeElapsed = timeNow - timeThen;

if (timeElapsed > 0) {
    ...      /* trigger synchronous events based on timeElapsed */
    ...      /* render model */
    timeThen = timeNow;
}
```

³Foolishly for TOS, but luckily for us.

Background: How to Detect Keyboard Input

TOS provides a `Cconis` system call for checking whether or not keyboard input is available for reading. And, it provides a `Cnecin` system call for actually reading it (without echoing the key press to the screen). Code like the following would read and process a key press only if one was pending:

```
#include <osbind.h>

...

if (Cconis()) {           /* is there user input read to be read? */
    char ch = (char)Cnecin(); /* if so, read it right away (otherwise don't */
    ...                /* waste time waiting for any) */
}
```

If `Cnecin` is called without first checking if a key has been pressed, it might *block* until a key is pressed.

More information on these TOS system calls is available in the reference subfolder of the course resources folder. The Devpac manual is a particular useful source of system call information.

Note: Some games require that key press and key release events be detected separately. Although this will be possible later, the routines above do not allow for this functionality, so these only key press events can be detected for now. However, TOS will cause a key that has been pressed and held down to auto-repeat. So, until a later stage, detection of auto-repeated key presses must temporarily be used in place of detection of a key release.

A Note on Detecting Mouse Input

The core game, as implemented in this checkpoint, is expected to be keyboard-driven. A welcome splash screen with a mouse-driven main menu will be added later. Therefore, mouse input detection is not normally required at this stage. Students who would like to incorporate mouse input now must consult with the instructor for more information.

DOUBLE BUFFERED GRAPHICS

Double buffered graphics will synchronize your game's frame rate (i.e. the rate at which successive frames of animation are produced) with the physical refresh rate of the video hardware and monitor. The goal is to achieve animated graphics that are free of visual distortions like flickering and tearing using a the *double buffering* technique.

Background: The Design of Video Games, Part 3

In computer animation, flickering and tearing occur when the frame rate is not synchronized with the refresh rate of the monitor. In other words, the video hardware is scanning the frame buffer at the same time as the frame buffer contents are being modified by the rendering routine.

To avoid this problem, it is necessary that these two processes occur at the same frequency (or that one frequency is a multiple of the other), and that they are in phase (i.e. that one doesn't start partway through the other).

One possible solution is to perform all the rendering for frames between screen refreshes, i.e. during the vertical blank intervals of a CRT monitor. This works well when the rendering process is very fast and can be guaranteed not to exceed the duration of the vertical blank (after which time the video hardware will begin scanning the frame buffer anew). However, this guarantee can be problematic in practice.

A better solution is to use the *double buffering* technique. A static image is held in the frame buffer while the next frame is rendered to a second "back buffer", off screen. Once this frame is complete, the *pages* are *flipped* at the next vertical blank. The former back buffer becomes the front buffer scanned by the video hardware during the next refresh, whereas the former front buffer can be used as the back buffer for rendering the next frame of animation.

It is important to note that the contents of the back buffer are *not copied* to the front buffer⁴. Instead, the video hardware register which contains the frame buffer start address is simply updated. The video hardware obtains the frame buffer start address from this video base register at each vertical blank, so that it is ready to start its next scan of the frame buffer at the correct location.

The use of double buffering does not guarantee smooth graphics. For example, if the rendering process was slow, at about 10 fps, the Atari ST's video hardware would perform about 7 screen refreshes for each page flip. Therefore, the animation would appear jumpy, albeit solid. In this scenario, effort would then need to be invested in optimizing the rendering process so that the animation was closer to (or ideally equal to) 70 fps.

⁴Copying an entire frame of memory can be as slow as rendering a frame, or slower if the rendering is optimized.

Requirement: Double Buffering

Double buffering must be implemented correctly, with absolutely no flickering or tearing. In this phase, all changes must be made to the main game module only.

First, it is necessary to allocate a 32,000 byte buffer. For hardware reasons that will be discussed later, frame buffers on the Atari ST must be 256-byte aligned (i.e. the start address must be a multiple of 256_{10})⁵. Initially, this buffer will serve as the back buffer (until the first flip).

Next, research the `Setscreen` system call (TOS XBIOS system call #5). This is the TOS routine which must be used to alter the frame buffer start address: it updates the video base register consulted by the video hardware at each vertical blank. Your main game loop must call `Setscreen` to schedule the flipping of the current front and back buffers each time the game program has finished rendering the next frame to the back buffer.

Common Problems

Stack smashing: the segment of memory reserved by TOS for the program's stack is not very large, and is less than 32,000 bytes wide. Therefore, the memory for the second frame buffer must *not* be allocated on the stack (i.e. as a local variable). This would cause the stack to overflow into the code segment. When the program began plotting to that frame buffer, it would corrupt existing code, resulting in odd behaviour and spurious crashes.

Incorrect parameters: In TOS, `Setscreen` can be used for more than one purpose. It takes three parameters, but only one of them (the physical frame buffer start address) is relevant to this project. The others should be passed as -1 to indicate that they are not being changed.

Premature rendering: `Setscreen` does *not* flip the pages immediately. When called, it updates the video base register so that the flip will happen at the next vertical blank, and then it returns immediately. Your code must therefore wait before starting to render the next frame. How long? Until after the next vertical blank has occurred⁶. If you forget this important detail and start rendering the next frame too soon, into what you think is the back buffer, you will still see flickering or tearing. This is because the buffer you *think* is the back buffer is really still the frame buffer, at least for a brief period of time.

Crash-on-quit: upon termination, the program must restore the original frame buffer. If not, 50% of the time, the program-allocated buffer will be at the front when TOS regains control. This region of memory is not where it expects the frame buffer to be, and strange behaviour and spurious crashes may result.

⁵Hint: allocate a region of memory slightly larger than 32,000 bytes, and then find an appropriate buffer start address within that range.

⁶Hint: it's easy to check this using the 70Hz *vertical blank counter*, i.e. the TOS-maintained longword at address 0x462 (**note:** this is the address used earlier). This counter is incremented at each vertical blank.

A Note on Achieving Smooth Animation at a High Frame Rate

The easiest way to implement double buffering is to erase the entire back buffer, and to fully render each frame, as described in Checkpoint 2 without completing the *Minimizing Screen Plotting* requirement; however, since this is the least time-efficient solution, the game will very likely not achieve a high enough frame rate. In other words, the animation will be flicker and tear-free, but may be *jumpy*. It is usually faster (although more complicated) to only re-plot the portions of the buffer which have changed. The problem when double buffering, though, is that the back buffer to be updated contains the view of the world from *two* frames ago.

One solution is as follows: right before rendering each frame, record a *snapshot* of the relevant parts of the model. The frame of animation in each buffer is paired with its corresponding snapshot, even though the model continues to evolve. When it comes time to render the next frame to the current back buffer, the current state of the world can be compared to the earlier snapshot to determine which portions of the buffer need to be re-plotted.

Other Requirements

Your code must be highly readable and self-documenting, including proper indentation and spacing, descriptive variable and function names, etc. No one function should be longer than approximately 25-30 lines of code – decompose as necessary. Code which is unnecessarily complex or hard to read will be severely penalized.

For each function you develop, write a short header block comment which specifies:

1. Its purpose, from the caller's perspective (if not perfectly clear from the name);
2. The purpose of each input parameter (if not perfectly clear from the name);
3. The purpose of each output parameter and return value (if not perfectly clear from the name);
4. Any assumptions, limitations or known bugs.

At the top of each source file, write a short block comment which summarizes the common purpose of the data structures and functions in the file. This should also include a normal ID header, but should also include team member names and the author's name.

You should add inline comments if you need to explain an algorithm or clarify a particularly tricky block of code, but keep this to a minimum.